

# IrrigaTech

## Sobre o Projeto

O projeto **IrrigaTech API** foi desenvolvido com o objetivo de criar um sistema IoT de monitoramento e irrigação inteligente de plantas.

A aplicação permite:

- Cadastro de usuários
- Cadastro de plantas
- Registro de leituras de sensores
- Controle de irrigações automáticas e manuais
- Integração com banco de dados MySQL
- Comunicação via API REST

O sistema foi desenvolvido utilizando:

- Java
- Spring Boot
- MySQL
- JPA/Hibernate
- Maven
- Postman
- GitHub

---

## Objetivo do Sistema

O sistema simula uma solução de automação residencial/agricultura inteligente.

A ideia principal é:

1. Monitorar a umidade do solo
  2. Registrar dados de sensores
  3. Acionar irrigações automáticas
  4. Armazenar todo o histórico em banco de dados
-

# Tecnologias Utilizadas

| Tecnologia    | Função                      |
|---------------|-----------------------------|
| Java          | Linguagem principal         |
| Spring Boot   | Framework backend           |
| MySQL         | Banco de dados              |
| JPA/Hibernate | ORM e relacionamentos       |
| Maven         | Gerenciador de dependências |
| Postman       | Testes da API               |
| GitHub        | Versionamento do projeto    |

---

## Estrutura do Banco de Dados

O sistema possui 4 tabelas principais:

- usuario**

Responsável pelo cadastro dos usuários.

**Campos:**

- id
  - nome
  - email
  - senha
- 

- planta**

Responsável pelo cadastro das plantas monitoradas.

**Campos:**

- id
  - nome
  - especie
  - localizacao
  - umidadeMinima
  - irrigacaoAutomatica
  - usuario\_id
-

- **leitura\_sensor**

Responsável por armazenar os dados enviados pelos sensores.

**Campos:**

- id
  - umidade
  - temperatura
  - dataHora
  - planta\_id
- 

- **irrigacao**

Responsável por armazenar o histórico de irrigações.

**Campos:**

- id
  - automatica
  - duracaoSegundos
  - dataHora
  - planta\_id
- 

## Relacionamentos

O projeto utiliza relacionamentos JPA/Hibernate.

### Relacionamentos:

```
Usuario
  ↓
Planta
  ↓
LeituraSensor
  ↓
Irrigacao
```

**Explicação:**

- Um usuário pode possuir várias plantas
  - Uma planta pode possuir várias leituras de sensores
  - Uma planta pode possuir várias irrigações
-

# Estrutura do Projeto

```
src
├── main
│   ├── java
│   │   └── com.raiztech.raiztech_api
│   │       ├── controller
│   │       ├── entity
│   │       ├── repository
│   │       └── RaiztechApiApplication.java
│   └── resources
│       └── application.properties
```

---

## Endpoints da API

### Usuários

#### Listar usuários

GET /usuarios

#### Criar usuário

POST /usuarios

#### Exemplo JSON:

```
{
  "nome": "Bruna",
  "email": "bruna@email.com",
  "senha": "123456"
}
```

---

### Plantas

#### Listar plantas

GET /plantas

#### Criar planta

POST /plantas

#### Exemplo JSON:

```
{
  "nome": "Samambaia",
  "especie": "Samambaia Americana",
}
```

```
"localizacao": "Sala",
"umidadeMinima": 40,
"irrigacaoAutomatica": true,
"usuario": {
  "id": 1
}
}
```

---

## Leituras de Sensor

### Listar leituras

GET /leituras

### Criar leitura

POST /leituras

### Exemplo JSON:

```
{
  "umidade": 42.5,
  "temperatura": 25.0,
  "planta": {
    "id": 1
  }
}
```

---

## Irrigações

### Listar irrigações

GET /irrigacoes

### Criar irrigação

POST /irrigacoes

### Exemplo JSON:

```
{
  "automatica": true,
  "duracaoSegundos": 15,
  "planta": {
    "id": 1
  }
}
```

---

## Funcionamento do Sistema

O sistema simula o funcionamento de um vaso inteligente.

### Fluxo:

1. O sensor lê a umidade do solo
2. O ESP32 enviaria os dados para a API
3. A API registra os dados no MySQL
4. O sistema verifica a umidade mínima
5. Caso necessário, a irrigação é acionada
6. O histórico fica salvo no banco

---

## Componentes Utilizados no Projeto IoT

| Componente                   | Função                     |
|------------------------------|----------------------------|
| ESP32 NodeMCU                | Microcontrolador principal |
| Sensor Capacitivo de Umidade | Leitura da umidade do solo |
| Mini Bomba Submersível       | Irrigação automática       |
| Módulo Relé                  | Acionamento da bomba       |
| Mangueira e fios jumper      | Conexões                   |
| Fita LED Grow                | Iluminação                 |

---

## Testes Realizados

Os testes foram realizados utilizando o Postman.

Foram testados:

- Cadastro de usuários
- Cadastro de plantas
- Cadastro de leituras
- Cadastro de irrigações
- Persistência no banco MySQL
- Relacionamentos entre tabelas

---

## Resultados Obtidos

O sistema conseguiu:

- Integrar Java com MySQL
- Criar API REST funcional
- Persistir dados no banco

- Realizar relacionamentos entre entidades
  - Simular um sistema IoT real
- 

## Conclusão

O projeto RaizTech API demonstrou na prática o desenvolvimento de um sistema backend utilizando Java Spring Boot integrado ao MySQL.

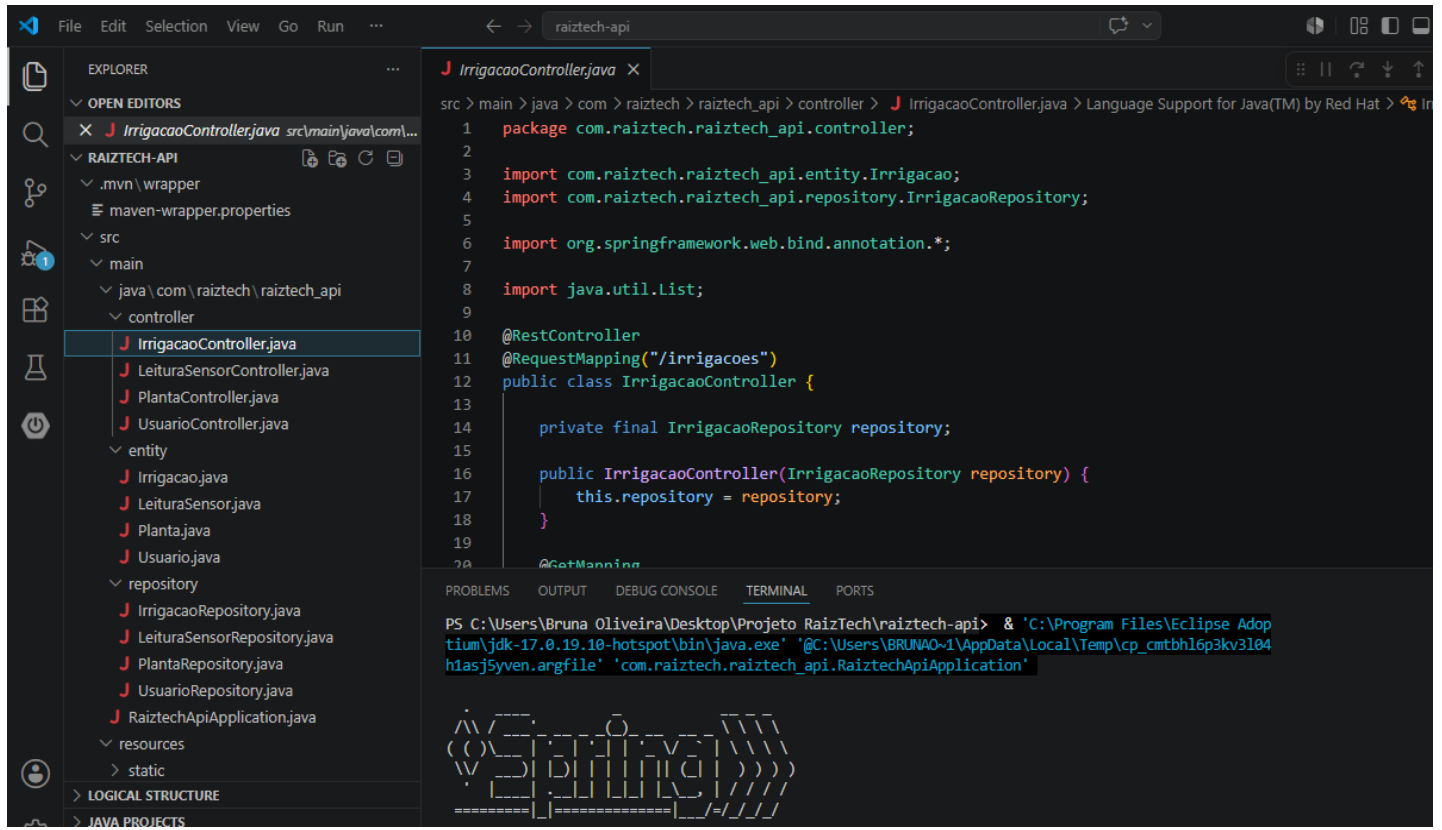
Além disso, o projeto simulou um ambiente IoT para monitoramento inteligente de plantas, permitindo registrar leituras de sensores e controlar irrigações.

O sistema pode ser expandido futuramente com:

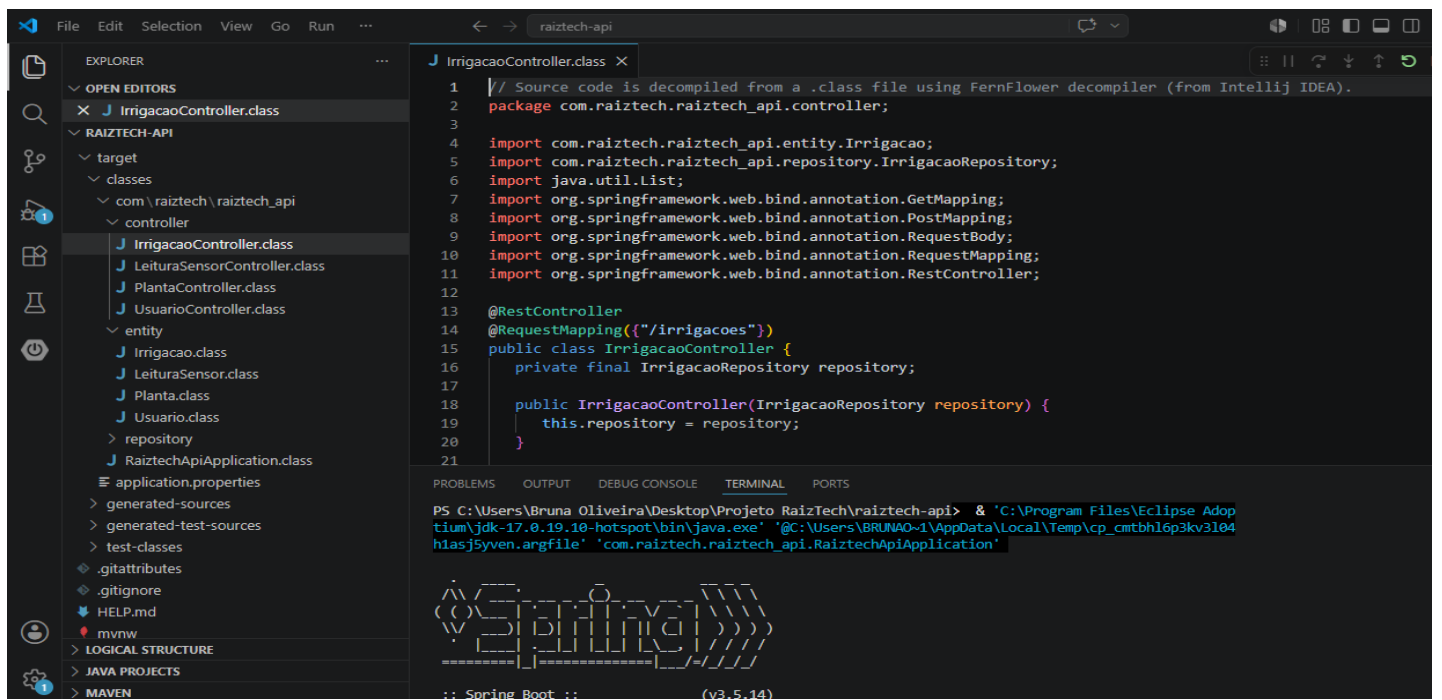
- Front-end web/mobile
- Integração real com ESP32
- Dashboard de monitoramento
- Gráficos em tempo real
- Login JWT
- Notificações automáticas

## Telas: VS CODE

Estrutura: controller, entity, repository e resources

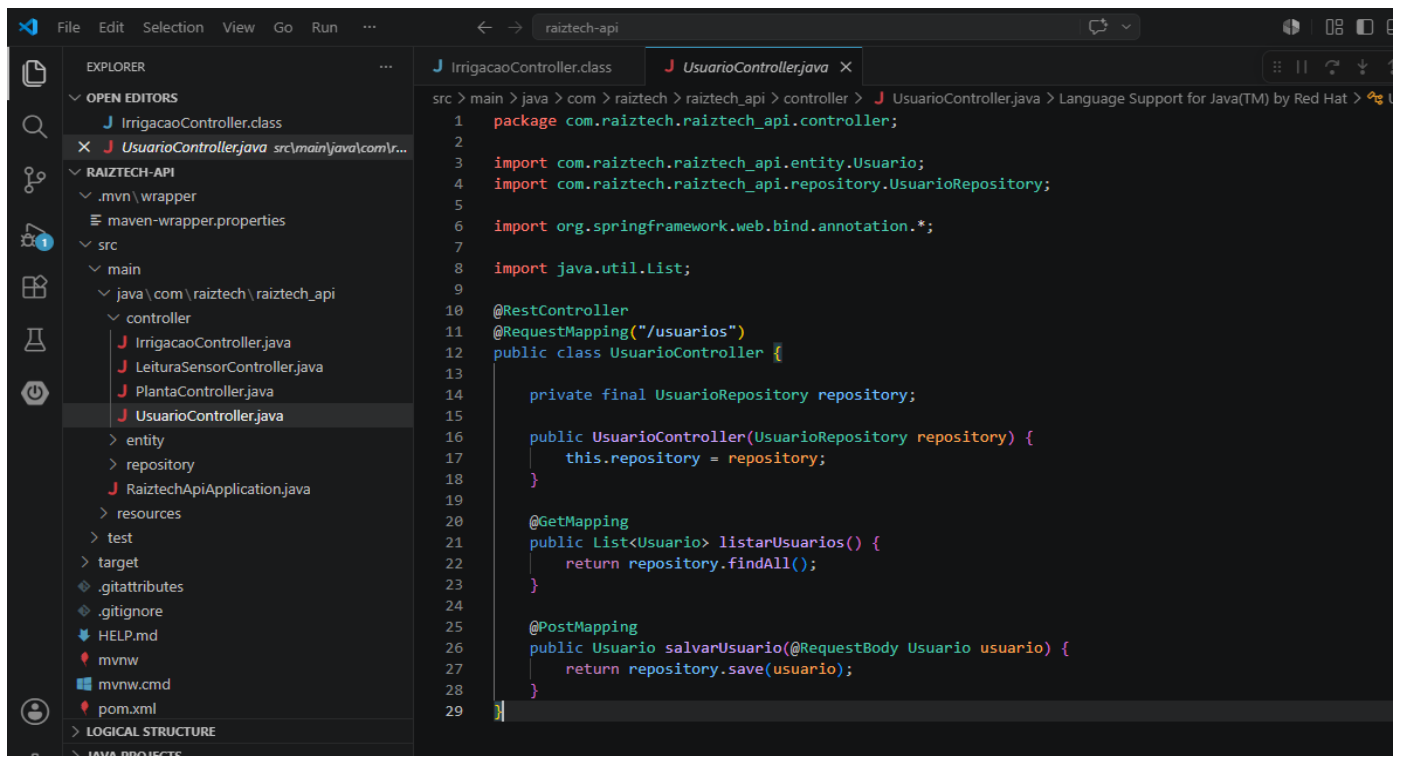


Estrutura Class: controller, entity, repository e resources

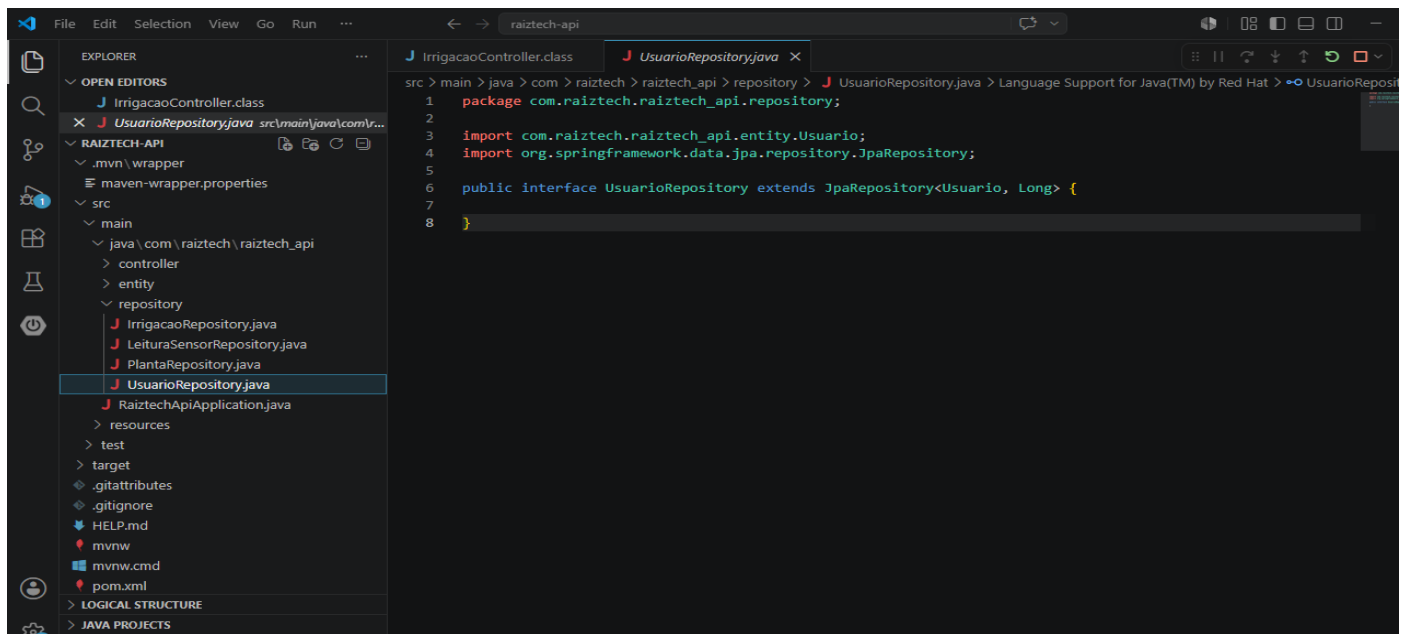




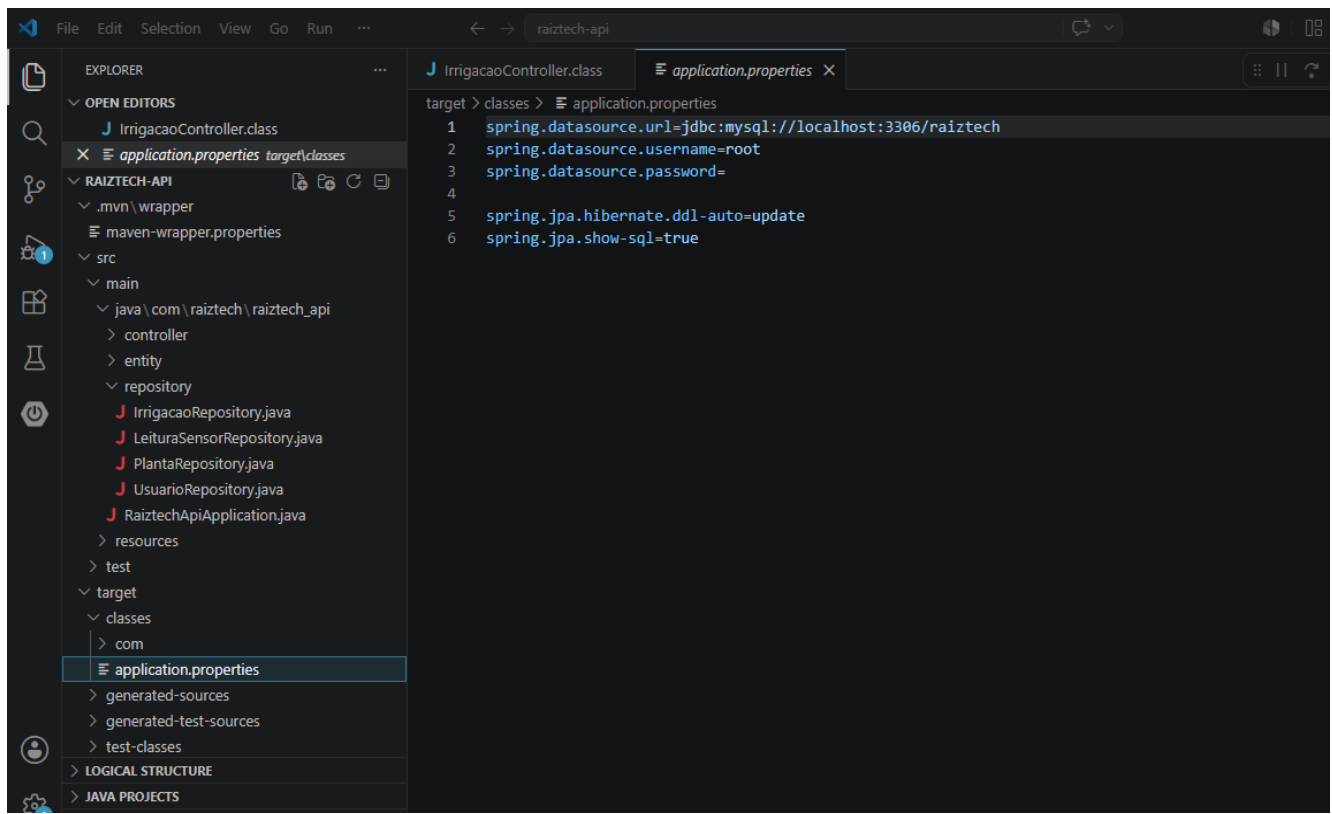
## Estrutura API REST



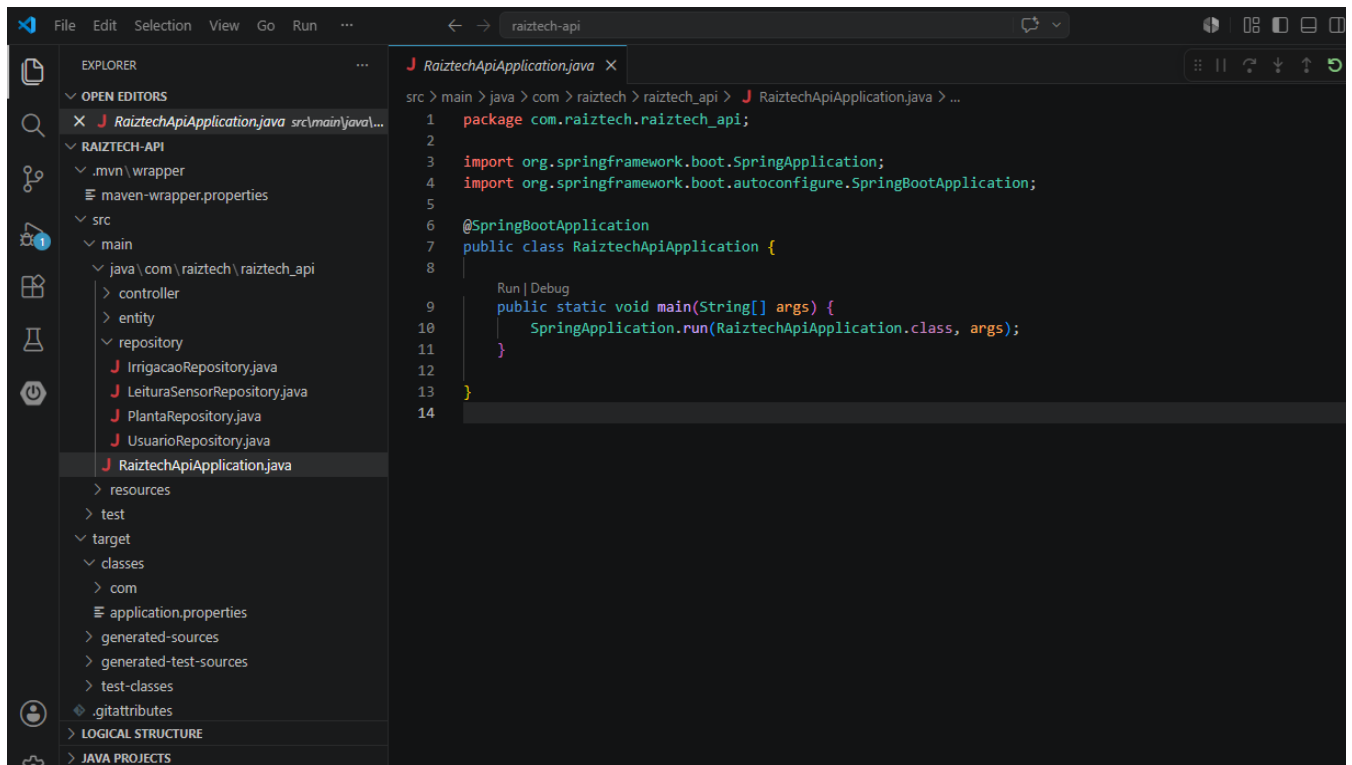
## Integração com o Banco



## Informações do Banco



## Aplicação Java Rodando



## Telas: Banco de Dados MySql

**Tabela usuarios**, colunas: id, nome, email, senha, foto\_perfil e criado\_em

The screenshot displays a database management interface. On the left, a tree view shows the 'usuario' table with columns: id, nome, email, senha, foto\_perfil, and criado\_em. Below this, the 'Table: usuario' section lists the column data types: id (bigint AI PK), nome (varchar(255)), email (varchar(150)), senha (varchar(255)), foto\_perfil (varchar(255)), and criado\_em (timestamp). The main area shows the 'Result Grid' for the 'usuario' table, listing 6 rows of data. Below the grid, the 'Output' section shows the 'Action Output' for a query: 'SELECT \* FROM usuario LIMIT 0, 1000', which returned 6 rows.

|   | id   | nome    | email             | senha  | foto_perfil | criado_em           |
|---|------|---------|-------------------|--------|-------------|---------------------|
| 1 | 1    | Bruna   | bruna@email.com   | 123456 | NULL        | 2026-05-19 11:29:52 |
| 2 | 2    | Felipe  | felipe@email.com  | 123456 | NULL        | 2026-05-19 14:16:49 |
| 3 | 3    | William | william@email.com | 123456 | NULL        | 2026-05-19 14:17:09 |
| 4 | 4    | Leticia | Leticia@email.com | 123456 | NULL        | 2026-05-19 14:17:25 |
| 5 | 5    | Arieli  | Arieli@email.com  | 123456 | NULL        | 2026-05-19 14:18:19 |
| 6 | 6    | Matheus | Matheus@email.com | 123456 | NULL        | 2026-05-19 14:18:35 |
| * | NULL | NULL    | NULL              | NULL   | NULL        | NULL                |

| # | Time     | Action                               | Message           |
|---|----------|--------------------------------------|-------------------|
| 1 | 10:24:00 | SELECT * FROM irigacao LIMIT 0, 1000 | 1 row(s) returned |
| 2 | 10:24:31 | SELECT * FROM usuario LIMIT 0, 1000  | 6 row(s) returned |
| 3 | 11:17:39 | SELECT * FROM planta LIMIT 0, 1000   | 6 row(s) returned |
| 4 | 11:20:29 | SELECT * FROM usuario LIMIT 0, 1000  | 6 row(s) returned |

**Tabela planta**, colunas: id, nome, especie, localizacao, umidade\_minima, irrigacao\_automatica e usuario\_id

The screenshot displays a database management interface. On the left, a tree view shows the 'planta' table with columns: id, nome, especie, localizacao, umidade\_minima, irrigacao\_automatica, and usuario\_id. Below this, the 'Table: usuario' section lists the column data types: id (bigint AI PK), nome (varchar(255)), email (varchar(150)), senha (varchar(255)), foto\_perfil (varchar(255)), and criado\_em (timestamp). The main area shows the 'Result Grid' for the 'planta' table, listing 6 rows of data. Below the grid, the 'Output' section shows the 'Action Output' for a query: 'SELECT \* FROM planta LIMIT 0, 1000', which returned 6 rows.

|   | id   | nome      | especie             | localizacao | umidade_minima | irrigacao_automatica | usuario_id |
|---|------|-----------|---------------------|-------------|----------------|----------------------|------------|
| 1 | 1    | Samambaia | Samambaia Americana | Sala        | 40             | 1                    | NULL       |
| 2 | 2    | Orquídea  | Phalaenopsis        | Quarto      | 50             | 1                    | NULL       |
| 3 | 3    | Cacto     | Cactaceae           | Escritório  | 20             | 0                    | NULL       |
| 4 | 4    | Suculenta | Echeveria           | Cozinha     | 30             | 0                    | NULL       |
| 5 | 5    | Jiboia    | Epipremnum aureum   | Varanda     | 45             | 1                    | NULL       |
| 6 | 6    | Lírio     | Lilium              | Sala        | 50             | 1                    | 1          |
| * | NULL | NULL      | NULL                | NULL        | NULL           | NULL                 | NULL       |

| # | Time     | Action                               | Message           |
|---|----------|--------------------------------------|-------------------|
| 1 | 10:24:00 | SELECT * FROM irigacao LIMIT 0, 1000 | 1 row(s) returned |
| 2 | 10:24:31 | SELECT * FROM usuario LIMIT 0, 1000  | 6 row(s) returned |
| 3 | 11:17:39 | SELECT * FROM planta LIMIT 0, 1000   | 6 row(s) returned |

**Tabela leitura\_sensor**, colunas: id, umidade, tempertura, data\_hora, planta\_id

The screenshot shows a database management interface. On the left, a tree view displays the database structure, including tables like 'leitura\_sensor', 'irrigacao', 'planta', 'usuario', and 'notificacao'. The 'leitura\_sensor' table is selected, and its schema is shown in the 'Table: leitura\_sensor' panel. The schema lists columns: id (bigint AI PK), umidade (double), temperatura (double), data\_hora (timestamp), and planta\_id (bigint). The 'Result Grid' panel shows a single row of data for the 'leitura\_sensor' table. The 'Output' panel shows the execution of a query, with a table of results including columns for #, Time, Action, and Message.

**Table: leitura\_sensor**

**Columns:**

- id: bigint AI PK
- umidade: double
- temperatura: double
- data\_hora: timestamp
- planta\_id: bigint

**Result Grid**

|   | id   | umidade | temperatura | data_hora           | planta_id |
|---|------|---------|-------------|---------------------|-----------|
| ▶ | 1    | 42.5    | 25          | 2026-05-19 14:46:24 | 1         |
| * | NULL | NULL    | NULL        | NULL                | NULL      |

**Output**

Action Output

| #   | Time     | Action                                     | Message           |
|-----|----------|--------------------------------------------|-------------------|
| ✓ 1 | 10:24:00 | SELECT * FROM irrigacao LIMIT 0, 1000      | 1 row(s) returned |
| ✓ 2 | 10:24:31 | SELECT * FROM usuario LIMIT 0, 1000        | 6 row(s) returned |
| ✓ 3 | 11:17:39 | SELECT * FROM planta LIMIT 0, 1000         | 6 row(s) returned |
| ✓ 4 | 11:20:29 | SELECT * FROM usuario LIMIT 0, 1000        | 6 row(s) returned |
| ✓ 5 | 11:24:39 | SELECT * FROM notificacao LIMIT 0, 1000    | 0 row(s) returned |
| ✓ 6 | 11:30:50 | SELECT * FROM leitura_sensor LIMIT 0, 1000 | 1 row(s) returned |

**Tabela irrigacao**, colunas: id, tipo, umidade\_antes, umidade\_depois, data\_hora, planta\_id, automatica, duracao\_segundos

The screenshot shows a database management interface. On the left, a tree view displays the database structure, including tables like 'irrigacao', 'leitura\_sensor', 'planta', 'usuario', and 'notificacao'. The 'irrigacao' table is selected, and its schema is shown in the 'Table: leitura\_sensor' panel. The schema lists columns: id (bigint AI PK), umidade (double), temperatura (double), data\_hora (timestamp), planta\_id (bigint), automatica (boolean), and duracao\_segundos (bigint). The 'Result Grid' panel shows a single row of data for the 'irrigacao' table. The 'Output' panel shows the execution of a query, with a table of results including columns for #, Time, Action, and Message.

**Table: leitura\_sensor**

**Columns:**

- id: bigint AI PK
- umidade: double
- temperatura: double
- data\_hora: timestamp
- planta\_id: bigint

**Result Grid**

|   | id   | tipo   | umidade_antes | umidade_depois | data_hora           | planta_id | automatica | duracao_segundos |
|---|------|--------|---------------|----------------|---------------------|-----------|------------|------------------|
| ▶ | 1    | manual | NULL          | NULL           | 2026-05-19 15:35:48 | 1         | 1          | 15               |
| * | NULL | NULL   | NULL          | NULL           | NULL                | NULL      | NULL       | NULL             |

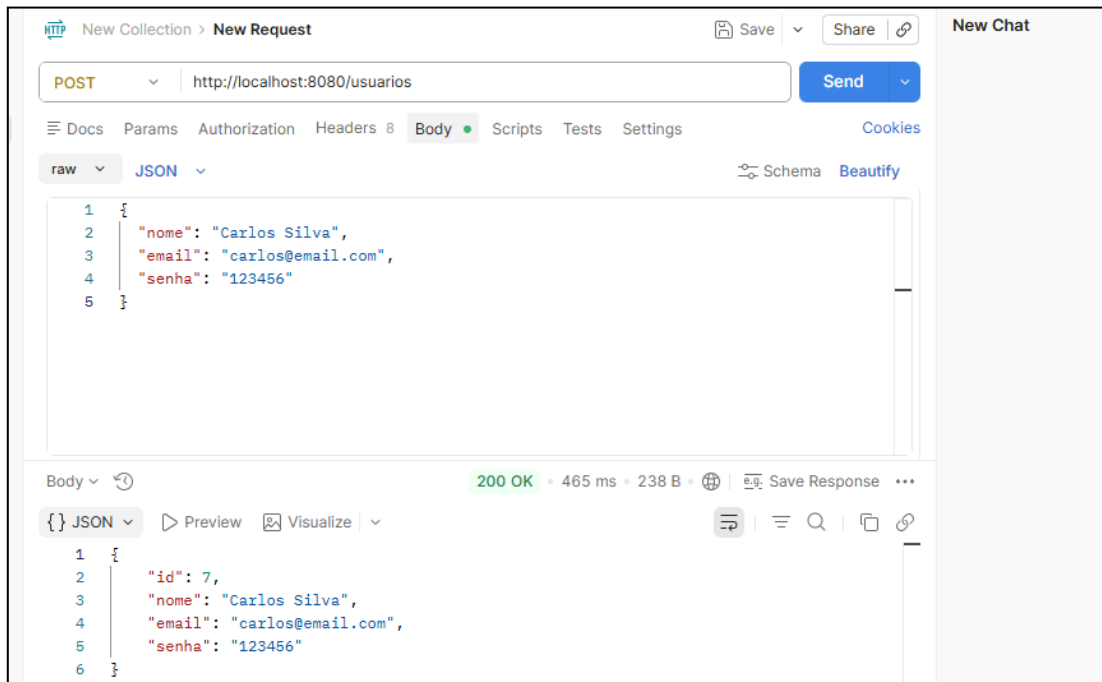
**Output**

Action Output

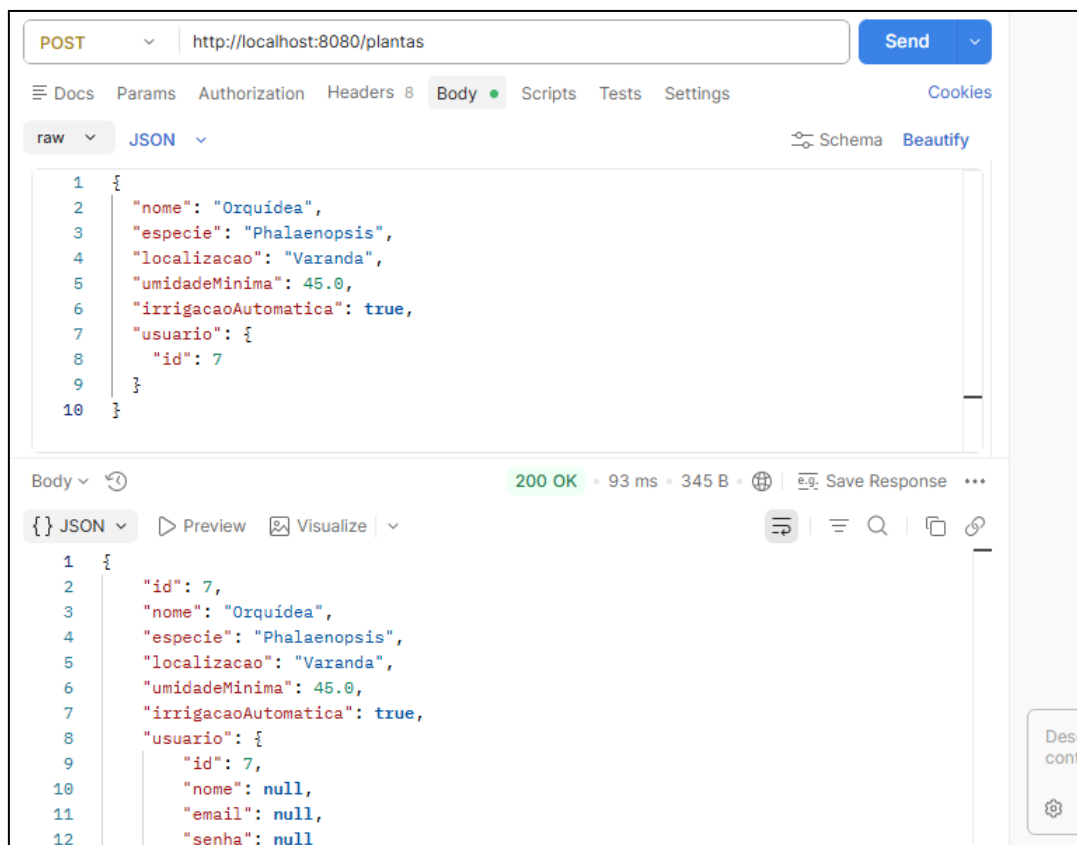
| #   | Time     | Action                                     | Message           |
|-----|----------|--------------------------------------------|-------------------|
| ✓ 2 | 10:24:31 | SELECT * FROM usuario LIMIT 0, 1000        | 6 row(s) returned |
| ✓ 3 | 11:17:39 | SELECT * FROM planta LIMIT 0, 1000         | 6 row(s) returned |
| ✓ 4 | 11:20:29 | SELECT * FROM usuario LIMIT 0, 1000        | 6 row(s) returned |
| ✓ 5 | 11:24:39 | SELECT * FROM notificacao LIMIT 0, 1000    | 0 row(s) returned |
| ✓ 6 | 11:30:50 | SELECT * FROM leitura_sensor LIMIT 0, 1000 | 1 row(s) returned |
| ✓ 7 | 12:00:20 | SELECT * FROM irrigacao LIMIT 0, 1000      | 1 row(s) returned |

## Telas: Postman (JSON) - Teste de API

Simulação de envio de usuário novo



Simulação de envio de planta nova cadastrando a um usuário



## Simulação de envio de leitura de umidade e temperatura

HTTP New Collection > New Request

POST http://localhost:8080/leituras

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "umidade": 32.5,
3   "temperatura": 26.0,
4   "planta": {
5     "id": 7
6   }
7 }
```

Body 200 OK • 60 ms • 374 B • Save Response

{ JSON Preview Visualize

```
1 {
2   "id": 2,
3   "umidade": 32.5,
4   "temperatura": 26.0,
5   "dataHora": "2026-05-21T12:20:58.2870234",
6   "planta": {
7     "id": 7,
8     "nome": null,
9     "especie": null,
10    "localizacao": null,
11    "umidadeMinima": null,
12    "irrigacaoAutomatica": null.

```

## Simulação de envio de irrigação a planta 7

HTTP New Collection > New Request

POST http://localhost:8080/irrigacoes

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "automatica": true,
3   "duracaoSegundos": 20,
4   "planta": {
5     "id": 7
6   }
7 }
```

Body 200 OK • 115 ms • 379 B • Save Response

{ JSON Preview Visualize

```
1 {
2   "id": 2,
3   "automatica": true,
4   "duracaoSegundos": 20,
5   "dataHora": "2026-05-21T12:18:30.1493139",
6   "planta": {
7     "id": 7,
8     "nome": null,
9     "especie": null,
10    "localizacao": null,
11    "umidadeMinima": null,
12    "irrigacaoAutomatica": null.

```